

Digest-Bound Replay Plans and State Declarations: Equivalence, Minimality, and Change Control (Series Synthesis)

Unified archive note v0.05 (2026-02-28; archive v132)

Abstract

The archive publishes brief, referee-grade anonymity papers while withholding large artifact bundles (logs, scripts, certificates) unless requested. This is only credible if (i) the public receipt surface commits to *which* replay workflow would validate the claim, and (ii) any long-lived system state assumptions are pinned in a comparable way.

This note standardizes two digest-bound adjuncts already used by the worked example: `plan_spec_id` (a canonical hash of the replay-plan specification) and `state_decl_id` (a canonical hash of the state-surface declaration). We then give a small, operational taxonomy for drift and equivalence: which changes are “publication-only” (re-run the verifier), and which changes are “claim-affecting” (recertify).

Non-redundancy note. Other papers in this archive should cite this note (Synthesis 18) whenever they need to justify why `plan_spec_id` and `state_decl_id` are digest-bound and which drift classes trigger replay vs recertification; they should not restate the taxonomy.

Scope. This is an interface/change-control note. It does *not* propose new privacy mechanisms. It exists so other papers can stay tight: when a paper says “replay plan π ” or “state contract σ ,” it should mean a digest-bound object, not prose.

Imports. Identifier conventions are centralized in Synthesis 8.[9]

Contract-object vocabulary is imported from Synthesis 0.[1] Receipt line-item semantics are imported from Synthesis 12.[2] Trace-interface and threat-window identities are imported from Synthesis 3/4.[3, 4] The motivating end-to-end instantiation is the worked example (Synthesis 17).[5] State-dependent accounting motivation is imported from State 1.[7] Recertification policy under drift is imported from Certified C.[8]

Exports. (i) Minimal semantic cores for `plan_spec` and `state_decl`, (ii) digest-binding rules (`plan_spec_id`, `state_decl_id`), and (iii) a drift/equivalence taxonomy that tells deployers when to republish, when to re-run verifiers, and when to recertify.

1 Replay plans: what a receipt promises

Many papers export a numeric budget, but the public object that makes the budget meaningful is a *receipt line item*. [2] A line item cannot ship the full artifact bundle in a source-only release, so it must at least commit to: (a) *which* replay workflow would validate the claim, and (b) *what* evidence that workflow expects.

Definition 1 (Replay plan specification). *A replay plan specification `plan_spec` is a short, deterministic description of the replay workflow referenced by `replay_hook`. It should contain only semantics a third party must know to (i) request the right private artifacts and (ii) deterministically recompute the public outputs.*

Minimal semantic core. To minimize needless churn while remaining referee-grade, we recommend that a `plan_spec` include exactly:

1. **Owner and scope:** the paper/module that owns the replay semantics; the claim fields the plan is intended to validate (typically `exposure_nf_id`, `tw_id`, and optional `state_decl_id`).
2. **Inputs:** a list of required artifacts by name *and* a short input-contract statement (e.g., “trace bundle includes raw transcripts + eligibility filter outputs + randomness commitments”).
3. **Determinism:** the intended interpreter/toolchain version or a verifier binary digest.
4. **Checks:** a small list of named predicates the verifier applies (e.g., conformance, sample-selection, accountant recomputation).
5. **Outputs:** the expected public outputs (e.g., `verifier_report.json` schema hash; receipt acceptance bit; derived OINL fields).

Avoid timestamps, hostnames, and other non-semantic metadata. If deployments want to ship operator notes, they should be separate, non-hashed fields.

Definition 2 (Digest-binding for replay plans). *Define*

$$plan_spec_id := sha256(canon(plan_spec)),$$

where *canon* is a deterministic canonicalization (sorted keys, no whitespace, stable number formatting). A receipt line item that includes `replay_hook.plan_id` should also include `replay_hook.plan_spec_id` so “same plan id, different replay” cannot hide off-surface.[2, 5]

Remark 1 (Plan certificates and transparency logs are replay evidence). *A `plan_spec_id` can also bind operational commitments that live outside the core anonymity math: for example, a schedule-only deployment can publish a signed plan certificate and an inclusion proof in a plan transparency log (PTL) and treat those objects as named inputs to the replay workflow. Operational B provides a concrete instantiation of this pattern.[6]*

Remark 2 (Why this does not overreach). *`plan_spec_id` does not certify the privacy claim. It certifies the replay promise: what an auditor will do if they request the evidence. This is the minimum needed for non-equivocation and mechanical diffing of receipts across releases.*

2 State declarations: what state a bound assumes

Many anonymity claims are implicitly stateful (caches, reputation, long-lived monitors, adaptive fallback gates). When a bound depends on such state, a human label like `state_contract_id` is not a stable interface: it can drift without changing the label.

Definition 3 (State-surface declaration). *A state declaration `state_decl` is a signed adjunct that commits to which long-lived control-plane state surfaces are in scope for the claim. It should be treated as part of the witness interface for stateful deployments.[7, 5]*

Minimal semantic core. A `state_decl` should include:

1. **Surface inventory:** a list of named state surfaces (e.g., “fallback usage counters,” “cache eviction policy,” “monitoring summaries”) with explicit include/exclude flags.
2. **Retention and reset:** the reset unit (epoch, window) and retention horizon for each included surface.
3. **Read/write actors:** which roles can write the surface (router, monitor, operator) and what is considered public leakage vs. internal state.
4. **Binding to the claim window:** the intended `tw_id` (or family) the declaration is meant to accompany.

As with plans, avoid non-semantic metadata.

Definition 4 (Digest-binding for state surfaces). *Define*

$$\text{state_decl_id} := \text{sha256}(\text{canon}(\text{state_decl_core})),$$

where `state_decl_core` is the minimal semantic core and `canon` is a deterministic canonicalization. Receipts for stateful claims should carry `state_decl_id`. [2, 5]

Lemma 1 (Conditioning on a published state witness bounds joint leakage). *Let x, x' be two secrets (e.g., two lookup targets) and let (W, Y) be the pair of (i) a state witness $W = g(S)$ derived from internal state S and (ii) a published view Y (receipt line items, exposure-normal-form projections, or replay outputs). Write P_x for the joint law of (W, Y) under secret x and $P_{x'}$ analogously. Then*

$$\text{TV}(P_x, P_{x'}) \leq \text{TV}(P_x^W, P_{x'}^W) + \sup_w \text{TV}(P_x^{Y|W=w}, P_{x'}^{Y|W=w}).$$

In particular, if W is public and independent of the secret (e.g., a public refresh timer or a public “bucket-staleness” class), then the joint distinguishing advantage of (W, Y) is controlled by the worst-case conditional leakage of Y given W .

Proof. For each w, y ,

$$P_x(w, y) - P_{x'}(w, y) = (P_x(w) - P_{x'}(w))P_x(y | w) + P_{x'}(w)(P_x(y | w) - P_{x'}(y | w)).$$

Summing absolute values over (w, y) and using $\sum_y P_x(y | w) = 1$ gives $\|P_x - P_{x'}\|_1 \leq \|P_x^W - P_{x'}^W\|_1 + \sup_w \|P_x^{Y|w} - P_{x'}^{Y|w}\|_1$. Divide by 2 to obtain the claim. $\square$

Remark 3 (How to use this lemma in the series). *State declarations pin which state surfaces are in scope. When a proof actually conditions on a public state summary (rate-limit class, bucket-staleness class, refresh epoch), publishing that summary as an explicit receipt line item (or as part of the exposure normal form) makes the dependence auditable, and Lemma 1 turns “statefulness” into a conditional-budget accounting step. See the state series for examples of when omitting such witnesses breaks naive horizon composition.* [7]

3 Equivalence and drift: an operational taxonomy

This section is deliberately operational: it tells a deployer what action is required when an id changes.

Change	Minimal required action
ENF-ID or TW-ID changes	New claim object (new receipt line item and ABOM/OINL); any reuse must be explicit via lineage/coarsening.[3, 4]
StateDecl changes (<code>state_decl_id</code> changes)	Treat as claim-affecting for stateful bounds: republish receipt/ABOM; if the new surface could increase leakage, trigger recertification.[7, 8]
PlanSpec changes (<code>plan_spec_id</code> changes)	Always republish the receipt/ABOM to pin the new replay promise. Whether the privacy claim must be recertified depends on whether the verifier/accountant semantics changed (taxonomy below).

Table 1: Digest-bound ids turn “is this the same claim?” into a mechanical diff.

Plan-spec drift classes (P0/P1/P2)

Let `plan_spec` change from `plan_spec_id` from h_0 to h_1 . Classify the change by what an auditor would compute on the same artifact bundle:

- **P0 (non-semantic):** the verifier output and acceptance predicate are unchanged (e.g., refactoring, documentation, performance). *Action:* republish the new `plan_spec_id`; no new privacy evidence is required.
- **P1 (checker-strengthening / determinism fixes):** acceptance may change only by rejecting previously-accepted bundles (more checks, tighter conformance, improved determinism). *Action:* republish and re-run the verifier on the old artifact bundle; if it still accepts, the claim can be reused.
- **P2 (claim-affecting semantics):** the accountant, sampling rule, or acceptance predicate changes in a way that could accept bundles that would previously be rejected, or could change the computed bound. *Action:* treat as recertification-relevant drift (Certified C): mint a new claim or run incremental/targeted audits as appropriate.[8]

This classification is intentionally conservative: without publishing full artifacts, the public guarantee is only as stable as the published replay semantics.

State-decl drift classes (S0/S1/S2)

Similarly, classify `state_decl` drift by whether the new declaration expands what can depend on the secret:

- **S0 (metadata-only):** renaming or rewording with no surface change.
- **S1 (surface tightening):** removing a surface from scope or reducing retention.
- **S2 (surface expansion):** adding a surface to scope, increasing retention, or allowing new actors to write secret-dependent state.

Action: S1 can often reuse the old bound conservatively; S2 should be treated as claim-affecting unless the owning state paper supplies an explicit attenuation lemma.

4 Checklist for keeping papers non-redundant

A mechanism paper should not define replay/state semantics in prose. Instead it should:

1. Export a receipt line item with `exposure_nf_id`, `tw_id`, and, when relevant, `state_decl_id`.
2. Point replay mechanics at `plan_id` *and* `plan_spec_id`.
3. When describing drift/updates, cite this note for the P0/P1/P2 and S0/S1/S2 rules rather than restating them.

References

- [1] Anonymous. Contract Objects for Anonymous-DHT Privacy Budgets and Claims. Synthesis 0, The-One-True-Archive (2026).
- [2] Anonymous. Receipt Line Items for Anonymity Claims: A Minimal Tuple Schema for Published Budgets. Synthesis 12, The-One-True-Archive (2026).
- [3] Anonymous. Trace Interfaces for Anonymous DHTs. Synthesis 3, The-One-True-Archive (2026).
- [4] Anonymous. Threat Models and Repetition Windows for Anonymous DHTs. Synthesis 4, The-One-True-Archive (2026).
- [5] Anonymous. Worked Example: Receipt/Interlock and Drift Comparison. Synthesis 17, The-One-True-Archive (2026).
- [6] Anonymous. Auditable Trace Privacy with Abort: Disclosure Receipts for Side-Channel Resistant Transport. Operational series Paper B, 2026. (This archive.)
- [7] Anonymous. State-Dependent Anonymity Budgets for Anonymous DHTs. AnonDHT State series Paper 1, The-One-True-Archive (2026).
- [8] Anonymous. Disagreement-Gated Recertification and Boundary Audits for Anonymous DHT Routers. Certified series Paper C, The-One-True-Archive (2026).
- [9] Anonymous. Notation and Minimal Model for the One-True-Archive. Series synthesis note (Synthesis 8), The-One-True-Archive (2026).